

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

РУБЕЖНЫЙ КОНТРОЛЬ 1

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-34Б _____ Корнеев Г.И.
(Подпись) (Ф.И.О.)

Проверил: _____ Гапанюк Ю.Е.
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

2026

Вариант запросов B

Предметная область 33 - Строка данных и таблица данных

Классы и связи

Класс DataRow хранит идентификатор строки, её название, количество заполненных значений и идентификатор основной таблицы. Класс DataTable хранит идентификатор и название таблицы. Класс DataRowTable содержит пары идентификаторов для связи многие-ко-многим.

Связь один-ко-многим задаётся полем table_id в объекте DataRow: у строки одна основная таблица, а в таблице может находиться несколько строк. Для связи многие-ко-многим используется список объектов DataRowTable, поэтому одну строку можно связать с несколькими таблицами.

Тестовые данные

Используются три таблицы, пять строк данных и восемь записей связи. Идентификаторы в связях соответствуют существующим объектам. Несколько строк связаны более чем с одной таблицей, что позволяет проверить запрос B3.

Класс	Поля	Назначение
DataRow	id, name, value_count, table_id	Строка данных и связь один-ко-многим
DataTable	id, name	Таблица данных
DataRowTable	row_id, table_id	Связь многие-ко-многим

Исходный код программы

main.py

```python
from operator import itemgetter


class DataRow:
    """Строка данных."""

    def __init__(self, id, name, value_count, table_id):
        self.id = id
        self.name = name
        self.value_count = value_count
        self.table_id = table_id


class DataTable:
    """Таблица данных."""

    def __init__(self, id, name):
        self.id = id
        self.name = name


class DataRowTable:
    """Связь строк и таблиц для отношения многие-ко-многим."""

    def __init__(self, row_id, table_id):
        self.row_id = row_id
        self.table_id = table_id


tables = [
    DataTable(1, "Основная таблица"),
    DataTable(2, "Архив"),
    DataTable(3, "Справочник"),
]

data_rows = [
    DataRow(1, "Альфа", 7, 1),
    DataRow(2, "Бета", 3, 1),
    DataRow(3, "Альманах", 5, 2),
    DataRow(4, "Гамма", 2, 2),
    DataRow(5, "Атрибут", 9, 3),
]

data_row_tables = [
    DataRowTable(1, 1),
    DataRowTable(1, 3),
    DataRowTable(2, 1),
    DataRowTable(3, 2),
    DataRowTable(3, 1),
    DataRowTable(4, 2),
    DataRowTable(5, 3),
    DataRowTable(5, 2),
]


def main():
    """Выполнение запросов варианта В."""

    # Соединение данных один-ко-многим по полю table_id.
    one_to_many = [
        (row.name, row.value_count, table.name)
        for table in tables
        for row in data_rows
        if row.table_id == table.id
    ]

    # Соединение данных многие-ко-многим через класс DataRowTable.
    many_to_many_temp = [
        (table.name, table.id, link.row_id)
        for table in tables
        for link in data_row_tables
        if table.id == link.table_id
    ]
    many_to_many = [
        (row.name, table_name)
        for table_name, table_id, row_id in many_to_many_temp
        for row in data_rows
        if row.id == row_id
    ]

    print("Задание В1")
    rows_starting_with_a = list(filter(lambda row: row[0].startswith("A"), one_to_many))
    res_11 = [(name, table_name) for name, _, table_name in rows_starting_with_a]
    print(res_11)

    print("\nЗадание В2")
    min_values_by_table = []
    for table in tables:
        table_rows = list(filter(lambda row: row[2] == table.name, one_to_many))
        if len(table_rows) > 0:
            values = [value_count for _, value_count, _ in table_rows]
            min_value = min(values)
            min_values_by_table.append((table.name, min_value))
    print(sorted(min_values_by_table, key=itemgetter(1)))

    print("\nЗадание В3")
    print(sorted(many_to_many, key=itemgetter(0)))


if __name__ == "__main__":
    main()
```

Результаты выполнения программы

Фактический вывод команды python3 main.py:

```
korneev@MacBook-Pro-Grigorij PCPL_RK-1 % python3 main.py
Задание В1
[('Альфа', 7, 'Основная таблица'), ('Альманах', 5, 'Архив'), ('Атрибут', 9, 'Справочник')]

Задание В2
[('Архив', 2), ('Основная таблица', 3), ('Справочник', 9)]

Задание В3
[('Альманах', 'Основная таблица'), ('Альманах', 'Архив'), ('Альфа', 'Основная таблица'), ('Альфа', 'Справочник'), ('Атрибут', 'Архив'), ('Атрибут', 'Справочник'
), ('Бета', 'Основная таблица'), ('Гамма', 'Архив')]
korneev@MacBook-Pro-Grigorij PCPL_RK-1 %
```